

BAB 6 Navigasi Aplikasi

6.1 Tujuan Pembelajaran

1. Memahami konsep dasar navigasi dalam aplikasi mobile.
2. Menjelaskan fungsi dan peran navigasi dalam pengelolaan antar halaman (screen).
3. Mengimplementasikan navigasi antar halaman menggunakan library navigasi pada Expo Router.
4. Mengelola perpindahan data antar halaman menggunakan parameter navigasi.
5. Membangun struktur aplikasi yang modular dan terorganisir dengan sistem navigasi.

6.2 Dasar Teori

Expo Router adalah sistem navigasi berbasis *file-based routing* untuk aplikasi React Native yang menggunakan Expo. Konsep ini mengadopsi pendekatan routing modern seperti yang digunakan pada framework web (misalnya Next.js), di mana struktur folder dan nama file secara otomatis menjadi rute (route) dalam aplikasi.

Berbeda dengan pendekatan konvensional seperti React Navigation yang memerlukan konfigurasi navigator secara manual, Expo Router menyederhanakan navigasi dengan memanfaatkan struktur direktori sebagai dasar sistem routing. Dimana pada expo router setiap file di dalam folder *app/* merepresentasikan satu screen (halaman) dalam aplikasi.

Expo Router digunakan untuk:

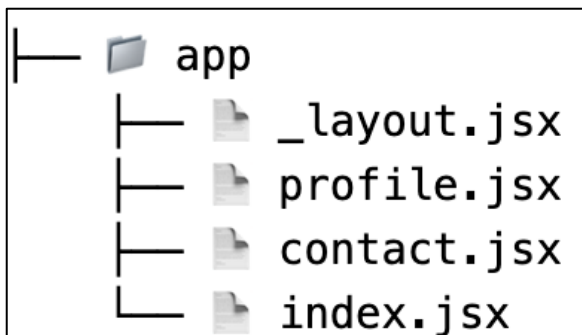
- Mengatur navigasi antar halaman secara otomatis berdasarkan struktur folder
- Menyederhanakan konfigurasi navigasi
- Mendukung navigasi bertingkat (nested navigation)
- Mengelola layout global dan layout per bagian aplikasi
- Mengimplementasikan dynamic routes (parameter dinamis)
- Meningkatkan keterbacaan dan skalabilitas project

6.2.1 Cara Kerja Expo Router

Expo Router bekerja berdasarkan beberapa prinsip utama:

1. Folder *app/* sebagai root routing

Semua file screen harus berada dalam folder *app/*



Mapping otomatis:

- *index.js* : merepresentasikan /
- *profile.js* : merepresentasikan */profile*
- *contact.js* : merepresentasikan */contact*

2. File adalah Screen

app/index.jsx : File tersebut otomatis menjadi halaman utama aplikasi.

app/_layout.jsx : digunakan untuk mengatur

struktur navigasi seperti Stack, Tabs, atau Drawer. Layout ini akan membungkus semua screen di dalam folder tersebut.

3. Navigasi menggunakan *useRouter*

Untuk berpindah halaman digunakan hook:

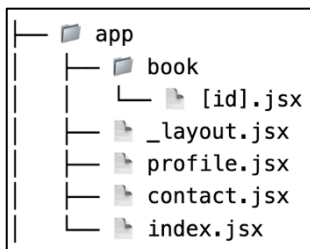
```
import { useRouter } from "expo-router";
const router = useRouter();
router.push("/profile");
```

Method navigasi:

- `router.push()` → menambahkan screen
- `router.replace()` → mengganti screen
- `router.back()` → kembali

4. Dynamic Routes (Parameter)

Expo Router mendukung parameter dinamis menggunakan tanda kurung siku [param].



Contoh Navigasi:

```
router.push("/book/10");
```

Cara mengambil parameter:

```
import { useLocalSearchParams } from "expo-router";
const { id } = useLocalSearchParams();
```

6.2.1 Struktur Navigasi pada Expo Router

Expo Router mendukung berbagai struktur navigasi melalui layout:

1. Stack Navigation

Stack navigator merupakan cara paling dasar dan fundamental untuk melakukan navigasi antar halaman (*route*) dalam sebuah aplikasi. Konsep yang digunakan adalah tumpukan (*stack*), di mana setiap halaman baru yang dibuka akan ditempatkan di atas halaman sebelumnya.

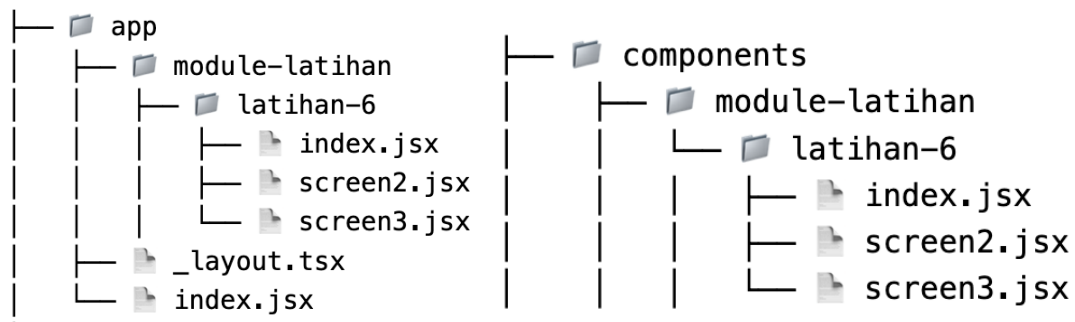
Perilaku Animasi pada Android dan iOS

Stack navigator juga menyesuaikan animasi transisi sesuai dengan platform:

- Android: Halaman baru akan muncul dengan animasi bertumpuk di atas layar sebelumnya (biasanya efek fade atau slide sesuai native behavior Android).
- iOS: Halaman baru akan muncul dengan animasi geser dari kanan ke kiri, mengikuti standar navigasi aplikasi iOS.

Hal ini membuat pengalaman pengguna terasa natural dan konsisten dengan sistem operasi perangkat.

Berikut ini adalah contoh penerapan Stack Navigation untuk berpindah halaman, dimana dalam contoh ini memiliki tiga buah screen dengan root layout:



Gambar 6.1 Struktur Navigasi Folder

App/index.jsx

```
import { Redirect } from "expo-router";

export default function Index() {
  return <Redirect href="/module-latihan/latihan-6" />;
}
```

Root Layout app/module-latihan/latihan-6:

index.jsx

```
export { default } from "@components/module-latihan/latihan-6/index";
```

screen2.jsx

```
export { default } from "@components/module-latihan/latihan-6/screen2";
```

screen3.jsx

```
export { default } from "@components/module-latihan/latihan-6/screen3";
```

RFC components latihan-6

index.jsx

```
import { Link, useRouter } from "expo-router";
import { Button, StatusBar, StyleSheet, Text, View } from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";

export default function Latihan6() {
  const router = useRouter();
  return (
    <SafeAreaView style={{ flex: 1 }}>
      <StatusBar barStyle={"dark-content"} />
      <View>
        <Text>First Screen</Text>
        <Link href="/module-latihan/latihan-6/screen2" push asChild>
          <Button title="Go to second screen" />
        </Link>
        <Button
          title="Go to thrid screen"
          onPress={() => router.push("/module-latihan/latihan-6/screen3")}
        />
      </View>
    </SafeAreaView>
  );
}
```

Pada komponen Latihan6, terdapat dua cara berbeda untuk melakukan navigasi di Expo Router:

1. Menggunakan `<Link>` Component

```
<Link href={"/module-latihan/latihan-6/screen2"} push asChild>
  <Button title="Go to second screen" />
</Link>
```

- `<Link>` adalah komponen deklaratif untuk navigasi
- `href`: Menentukan tujuan navigasi (path ke screen2)
- `push`: Menambahkan screen baru ke navigation stack (bisa kembali dengan tombol back)
- `asChild`: Membuat Link menggunakan child component (Button) sebagai trigger, bukan membuat wrapper tambahan

2. Menggunakan `useRouter()` Hook

```
const router = useRouter();
<Button
  title="Go to thrid screen"
  onPress={() => router.push("/module-latihan/latihan-6/screen3")}
/>
```

- `useRouter()` adalah hook yang memberikan akses ke router object
- `router.push()`: Method imperatif untuk navigasi programatik
- Dipanggil di dalam event handler (`onPress`)

screen2.jsx

```
import { useRouter } from "expo-router";
import { Button, StatusBar, StyleSheet, Text, View } from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";

export default function Screen2() {
  const router = useRouter();
  return (
    <SafeAreaView style={{ flex: 1 }}>
      <StatusBar barStyle={"dark-content"} />
      <View>
        <Text>Second Screen</Text>
        <Button title="Go back" onPress={() => router.back()} />
      </View>
    </SafeAreaView>
  );
}
```

`router.back()` :

- Method dari `useRouter()` hook
- Fungsi untuk navigasi kembali ke screen sebelumnya
- Menghapus screen saat ini dari navigation stack

screen3.jsx

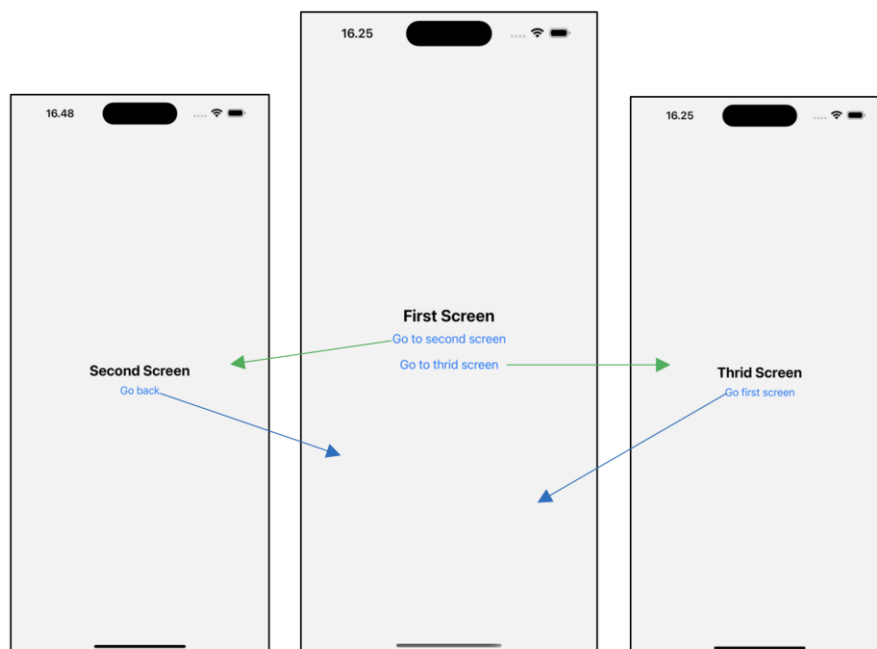
```
import { useRouter } from "expo-router";
import { Button, StatusBar, StyleSheet, Text, View } from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";

export default function Screen3() {
  const router = useRouter();
  return (
    <SafeAreaView style={{ flex: 1 }}>
      <StatusBar barStyle={"dark-content"} />
      <View>
        <Text>Thrid Screen</Text>
        <Button title="Go first screen" onPress={() => router.push("/")} />
      </View>
    </SafeAreaView>
  );
}
```

router.push("/") :

- *router.push()*: Method untuk navigasi ke screen baru
- *"/"*: Path/route tujuan navigasi, dimana '/' mengarah kepada komponen Latihan6 dimana komponen tersebut diinisialisasikan sebagai halaman utama (index.jsx)

Maka dari ketiga komponen diatas memiliki tampilan perpindahan screen seperti dibawah ini:



Gambar 6.2 Stack Navigation

Dengan menggunakan `<Stack />`, pengembang dapat menambahkan route baru ke dalam aplikasi tanpa harus membuat konfigurasi navigator secara manual seperti pada pendekatan tradisional.

2. Tab Navigation

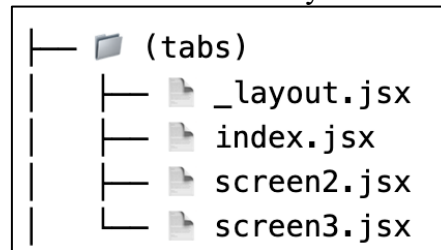
Tab Navigation adalah jenis navigasi yang menampilkan beberapa halaman utama dalam bentuk tab (menu berpindah), biasanya berada di bagian bawah layar (bottom tab). Setiap tab merepresentasikan satu route atau screen utama dalam aplikasi.

Pada Expo Router, Tabs disediakan melalui komponen `<Tabs/>` yang diambil dari library expo-router, `import { Tabs } from "expo-router"`. Penggunaan Tabs Navigation ini digunakan dalam file `_layout.jsx` untuk membungkus halaman-halaman yang dijadikan tab. Tabs memungkinkan pengguna berpindah antar halaman utama tanpa menghapus halaman sebelumnya dari stack.

Berbeda dengan Stack yang menumpuk halaman, Tabs bekerja dengan cara:

- Setiap tab memiliki route sendiri.
- Perpindahan tab tidak selalu menghapus screen sebelumnya.
- Setiap tab dapat memiliki stack navigasi sendiri (nested stack).

Struktur sederhananya seperti contoh code sebelumnya:



Langkah 1: Mengatur Layout Tabs

isi file `_layout.jsx`:

```

1  import { Ionicons } from "@expo/vector-icons";
2  import { Tabs } from "expo-router";
3
4  export default function TabLayout() {
5    return (
6      <Tabs
7        screenOptions={{
8          headerShown: false,
9          tabBarActiveTintColor: "#49745e",
10         tabBarInactiveTintColor: "gray",
11       }}
12     >
13       <Tabs.Screen
14         name="index"
15         options={{
16           title: "Screen 1",
17           tabBarIcon: ({ color, size }) => (
18             <Ionicons name="home" size={size} color={color} />
19           ),
20         }}
21       />
22       <Tabs.Screen
23         name="screen2"
24         options={{
25           title: "Screen 2",
26           tabBarIcon: ({ color, size }) => (
27             <Ionicons name="settings" size={size} color={color} />
28           ),
29         }}
30       />
31       <Tabs.Screen
32         name="screen3"
33         options={{
34           title: "Screen 3",
35           tabBarIcon: ({ color, size }) => (
36             <Ionicons name="alert-circle" size={size} color={color} />
37           ),
38         }}
39       />
40     </Tabs>
41   );
42 }
43

```

Penjelasan:

A. File-based Routing

- `<Tabs>` digunakan untuk menginisialisasi pembungkus untuk membuat navigasi dalam bentuk bottom tab
- `<Tabs.Screen>` diperuntukan untuk menentukan screen apa saja yang akan dimasukkan kedalam navigasi tab.
- Nama pada `<Tabs.Screen name="index">` harus sesuai dengan nama file
 - index → index.jsx
 - screen2 → screen2.jsx

- screen3 → screen3.jsx
- Sedangkan *options* digunakan untuk mengatur judul dan tampilan
 - props *title* digunakan untuk judul tab
 - props *tabBarIcon* digunakan jika navigasi tab ingin menggunakan sebuah icon
 - *color*: Otomatis berubah sesuai *tabBarActiveTintColor* atau *tabBarInactiveTintColor*
 - *size*: Ukuran default icon (biasanya 24)
 - *focused*: Boolean yang menunjukkan apakah tab sedang aktif (tidak digunakan di code ini)

B. Tab Navigation Flow

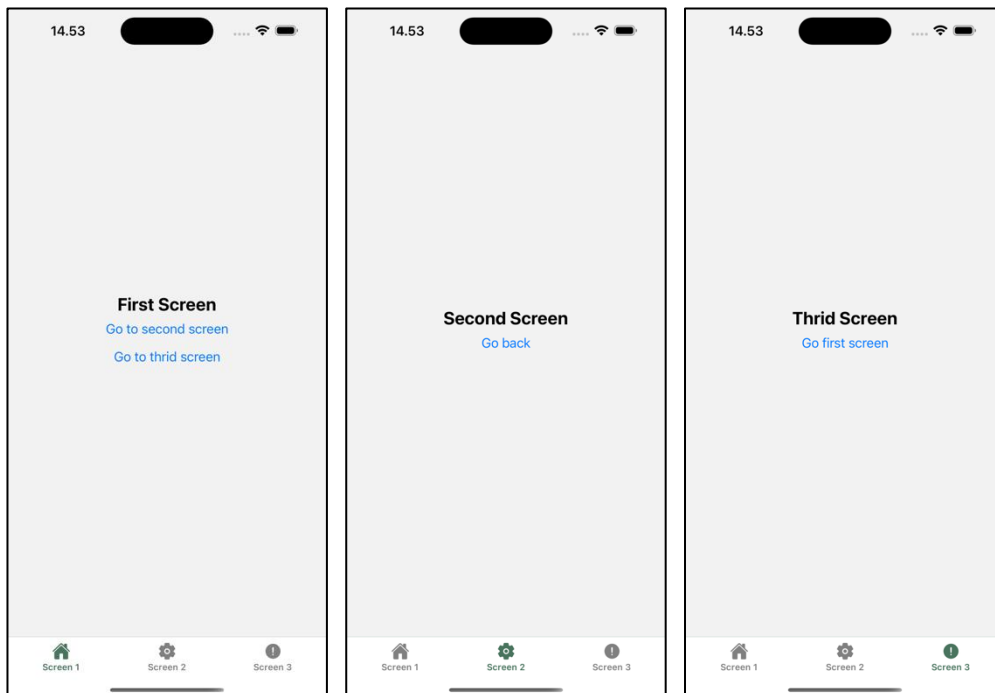
TabLayout (_layout.jsx)

- Tab 1: index.jsx (Home)
- Tab 2: screen2.jsx (Settings)
- Tab 3: screen3.jsx (Alert)

Langkah 2: Menambahkan Screen Tab

Untuk file index.jsx, screen2.jsx dan screen3.jsx tidak perlu diubah sama sekali, yang perlu dilakukan ialah menambahkan _layout.jsx seperti pada Langkah 1. Dan pastikan nama file harus sama pada Tab.Screen properti name pada.

Maka hasil yang didapatkan akan seperti gambar dibawah ini:



Gambar 6.3 Tab Navigations

Jika sedang berada di screen yang aktif maka tombol navigasi tab bottom akan berwarna hijau sesuai dengan pengaturan `<Tabs>` pada props *screenOptions* dengan nilai *tabBarActiveTintColor*: `"#49745e"`. Sedangkan sebaliknya nilai *tabBarInactiveTintColor*: `"gray"` akan dieksekusi jika sedang tidak aktif.

Keuntungan Menggunakan Tabs pada Expo Router:

- Struktur Lebih Rapi
Menggunakan pendekatan file-based routing sehingga tidak perlu konfigurasi navigator manual.
- Mudah Dikembangkan
Menambahkan tab cukup dengan menambahkan file baru dan mendefinisikannya di `_layout.jsx`.
- UX Lebih Baik
Cocok untuk aplikasi dengan beberapa fitur utama yang sering diakses.
- Mendukung Nested Navigation
Setiap tab bisa memiliki Stack sendiri.
- Minim Boilerplate Code
Tidak perlu `NavigationContainer` manual.

3. Drawer Navigation

Drawer Navigation adalah jenis navigasi yang menampilkan menu dalam bentuk panel geser (sidebar) dari sisi kiri atau kanan layar. Drawer biasanya berisi daftar halaman utama atau fitur penting dalam aplikasi.

Secara umum, drawer:

- Muncul ketika pengguna melakukan swipe dari sisi layar, atau
- Ketika pengguna menekan tombol menu (ikon ☰).

Pada Expo Router, Drawer merupakan navigasi berbasis file-system yang dibangun di atas React Navigation. Untuk menggunakan Drawer, Expo Router menyediakan komponen:

```
import { Drawer } from "expo-router/drawer";  
  
<Drawer />
```

Drawer didefinisikan dalam file `_layout.jsx`, sama seperti Stack dan Tabs.

Cara menggunakan Drawer Navigation:

Langkah 1: Install Dependency Tambahan

Drawer membutuhkan beberapa dependency tambahan karena menggunakan gesture dan animasi.

```
npx expo install @react-navigation/drawer react-native-reanimated react-native-worklets
```

Langkah 2: Mengatur Layout Drawer

isi file `layout.jsx`:

```
1 import { Drawer } from "expo-router/drawer";
2 import "react-native-reanimated";
3
4 export default function TabLayout() {
5   return (
6     <Drawer
7       screenOptions={{
8         headerShown: true,
9         drawerActiveTintColor: "#49745e",
10        drawerInactiveTintColor: "gray",
11        drawerStyle: {
12          backgroundColor: "#f8f6f1",
13          width: 250,
14        },
15        drawerLabelStyle: {
16          fontSize: 16,
17          fontWeight: "600",
18        },
19        headerStyle: {
20          backgroundColor: "#49745e",
21        },
22        headerTintColor: "white",
23        headerTitleStyle: {
24          fontWeight: "bold",
25        },
26      }}
27     >
28     <Drawer.Screen
29       name="index"
30       options={{
31         drawerLabel: "Screen 1",
32         title: "Screen 1",
33       }}
34     />
35     <Drawer.Screen
36       name="screen2"
37       options={{
38         drawerLabel: "Screen 2",
39         title: "Screen 2",
40       }}
41     />
42     <Drawer.Screen
43       name="screen3"
44       options={{
45         drawerLabel: "Screen 3",
46         title: "Screen 3",
47       }}
48     />
49   </Drawer>
50 );
51 }
```

Penjelasan:

A. File-based Routing

- `<Drawer>` digunakan untuk menginisialisasi pembungkus untuk membuat navigasi dalam bentuk bottom tab.
- `<Drawer.Screen>` diperuntukan untuk menentukan screen apa saja yang akan dimasukan kedalam navigasi tab.
- Nama pada `<Drawer.Screen name="index">` harus sesuai dengan nama file.

B. Opsi Konfigurasi `<Drawer />` pada props `screenOptions`

- Tampilkan header dengan hamburger icon : `headerShown: true`
- Warna text menu aktif : `drawerActiveTintColor: "#49745e"`
- Warna text menu tidak aktif: `drawerInactiveTintColor: "gray"`
- Mengatur Background dan lebar drawer :

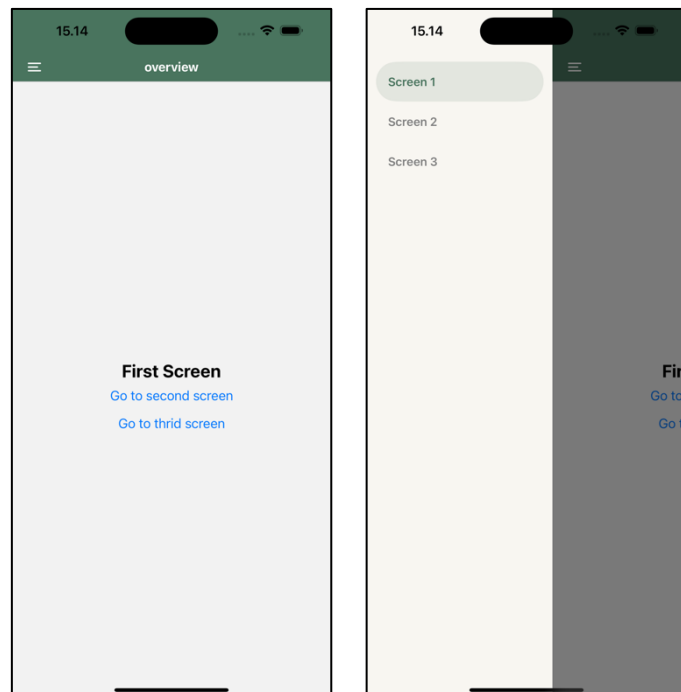
```
drawerStyle: {  
  backgroundColor: color_list.cream,  
  width: 250,  
}
```
- Mengatur Ukuran font drawer:

```
drawerLabelStyle: {  
  fontSize: 16, // Ukuran font label  
  fontWeight: "600", // Ketebalan font  
}
```
- Mengatur Background header:

```
headerStyle: {  
  backgroundColor: "#49745e"  
}
```
- Memberikan efek Warna icon dan text header:

```
headerTintColor: "#fff",  
headerTitleStyle: {  
  fontWeight: "bold",  
}
```

Setelah mengatur semua screen dan `_layout` maka tampilan dari Draw Navigation seperti dibawah ini:



Gambar 6.4 Drawer Navigations

Untuk mengakses menu navigasi yang muncul dari samping layar dibuka dengan swipe dari kiri ke kanan atau melalui icon burger ($\equiv$).

4. Nesting Navigation

Nesting navigation adalah teknik menggabungkan beberapa jenis navigasi dalam satu misalnya menggabungkan *Stack Navigator*, *Tabs Navigator*, dan *Drawer Navigator* agar struktur navigasi menjadi lebih modular dan fleksibel.

Pada Expo Router, nesting tidak dilakukan melalui konfigurasi manual yang rumit seperti pada React Navigation klasik, melainkan melalui struktur folder *nested* (bertumpuk) yang otomatis mendefinisikan navigasi bertingkat berdasarkan hierarki folder.

Expo Router membaca struktur folder di dalam direktori *app/* dan menginterpretasikannya sebagai hierarki navigasi. Dengan cara ini:

- Folder yang berada di root menjadi navigasi pertama (*stack root*).
- Folder berada di bawah folder lain → menjadi child navigasi (*nested*).
- File *_layout.jsx* dalam folder bertindak sebagai layout/parent yang membungkus children.

Struktur folder menjadi “konfigurasi” navigasi tanpa perlu menuliskan kode navigator secara eksplisit.

5. Pengiriman dan Pengambilan Parameter Antar Halaman

Dalam Expo Router, navigasi berbasis URL. Artinya, setiap halaman direpresentasikan sebagai path, dan data dapat dikirim melalui URL menggunakan parameter.

Parameter ini digunakan untuk:

- Menampilkan detail data tertentu (misalnya detail buku)
- Mengirim ID
- Mengirim filter pencarian
- Mengatur mode tampilan halaman

A. Jenis Parameter pada Expo Router

Secara umum, terdapat dua jenis parameter:

1. Route Parameters (Dynamic Route)

Digunakan ketika parameter menjadi bagian dari path URL.

Contoh:

/books/10

Struktur folder:

```
├─ app
│   └─ books
│       └─ [id].jsx
```

Tanda `[id]` menunjukkan bahwa `id` adalah parameter dinamis.

2. Search Parameters (Query Parameters)

Digunakan untuk mengirim data tambahan dalam bentuk query string.

Contoh:

```
/books?id=10&category=novel
```

Parameter dikirim setelah tanda ?.

B. Cara Mengirim Parameter

Pengiriman parameter dilakukan menggunakan:

- Link
- `router.push()`
- `router.replace()`

Contoh 1: Menggunakan Dynamic Route

```
<Pressable onPress={() => router.push("/books/10")}>  
  <Text>Lihat Detail Buku</Text>  
</Pressable>
```

Parameter yang dikirim:

id = 10

Contoh 2: Menggunakan Query Parameter

```
router.push({  
  pathname: "/books",  
  params: { id: 10, category: "novel" },  
});
```

URL yang terbentuk:

/books?id=10&category=novel

C. Cara Mengambil Parameter

Untuk membaca parameter pada halaman tujuan digunakan hook:

`useLocalSearchParams()`

Hook ini akan mengembalikan semua parameter dalam bentuk object.

Contoh:

Struktur folder:

```
|— app  
|   |— books  
|       |— [id].jsx
```

Code:

```
import { View, Text } from "react-native";
import { useLocalSearchParams } from "expo-router";

export default function DetailBuku() {
  const { id, category } = useLocalSearchParams();

  return (
    <View>
      <Text>ID Buku: {id}</Text>
      <Text>Kategori: {category}</Text>
    </View>
  );
}
```

D. Perbedaan Route Param dan Search Param

Aspek	Route Parameter	Search Parameter
Letak	Bagian dari path	Setelah tanda ?
Struktur File	Menggunakan [param]	Tidak perlu file khusus
Contoh	/books/5	/books?id=5
Umum Digunakan Untuk	Detail berdasarkan ID	Filter, sorting, pencarian

Tabel 6.1 Perbedaan Route Parameter dan Search Parameter

E. Perbedaan useLocalSearchParams dan useGlobalSearchParams

Expo Router menyediakan dua hook:

Hook	Fungsi
useLocalSearchParams()	Membaca parameter pada route aktif
useGlobalSearchParams()	Membaca parameter global pada seluruh navigasi

Tabel 6.2 Perbedaan search params

Pengiriman dan pengambilan parameter pada Expo Router dilakukan melalui mekanisme URL-based routing.

Komponen utama:

- Dynamic Route [param]
- Query Parameter ?key=value
- useRouter() untuk mengirim
- useLocalSearchParams() untuk membaca

Konsep ini sangat penting dalam:

- Nested Navigation
- Drawer Navigation

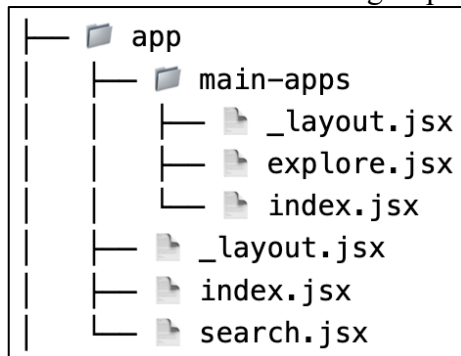
6.3 Praktikum

Pada latihan praktikum kali ini, akan melanjutkan pengembangan aplikasi *e-catalog* buku yang telah dibuat sebelumnya dengan mengimplementasikan konsep *nesting navigation* menggunakan Expo Router. Jika pada tahap awal aplikasi hanya menampilkan daftar buku secara sederhana, maka pada tahap ini struktur navigasi akan dikembangkan menjadi lebih terorganisir dan modular. Dengan memanfaatkan teknik *nested navigation*, setiap fitur seperti halaman daftar buku, detail buku, dan kemungkinan fitur tambahan lainnya akan

dikelompokkan dalam struktur folder yang saling terhubung secara hierarkis. Pendekatan ini tidak hanya meningkatkan keteraturan kode, tetapi juga memberikan pemahaman konseptual mengenai bagaimana navigasi bertingkat bekerja dalam pengembangan aplikasi berbasis React Native dan Expo. Melalui latihan ini, diharapkan mampu memahami relasi antar-layout, struktur folder, serta alur perpindahan antar-halaman dalam arsitektur navigasi yang lebih kompleks dan *scalable*.

Langkah 1: Membuat struktur navigasi pada *app/*

Buatlah skema struktur navigasi program e-catalog seperti dibawah ini:



_layout.jsx : diperuntukan untuk container pembungkus awal halaman dengan menggunakan expo-router tipe Stack Navigation, dimana container ini akan memanggil seluruh file yang ada didalam folder *app*.

index.jsx : dipersiapkan sebagai Landing Page untuk halaman awal tampilan aplikasi.

search.jsx : disiapkan untuk halaman pencarian item, dimana file ini diambil dari tugas praktikum pada bab sebelumnya.

main-apps/ : folder ini akan menyimpan seluruh halaman utama aplikasi seperti, home, profile, explore dan lainnya.

main-apps/_layout.jsx : container ini akan membungkus halaman utama aplikasi dengan menggunakan expo-router dengan tipe Tab Navigation.

main-apps/index.jsx : akan merepresentasikan halaman Home Screen, file ini diambil dari tugas praktikum pada bab sebelumnya.

main-apps/explore.jsx : halaman yang menampilkan tampilan sederhana berupa text.

Langkah 2: Mengatur container *_layout.jsx* dengan menerapkan Stack Navigation

_layout.jsx:

```
import { Stack } from "expo-router";
import { StatusBar } from "expo-status-bar";
import "react-native-reanimated";

export default function RootLayout() {
  return (
    <>
      <StatusBar style="auto" hidden />
      <Stack screenOptions={{ headerShown: false }} />
    </>
  );
}
```

Pada komponen RFC RootLayout diatas penggunaan <Stack /> dipakai untuk menandakan bahwa seluruh file yang ada pada folder *app/* akan disusun secara bertumpuk.

Langkah 3: Mengatur tampilan halaman utama aplikasi

Halaman utama aplikasi akan otomatis mengambil atau membaca code pada file *index.jsx* pada folder *app/*. Oleh karenanya pada file ini diperuntukan sebagai Landing Page aplikasi.

index.jsx:

```
import { Button, Text, View } from "react-native";

export default function Index() {
  return (
    <View>
      <Text>Landing Page</Text>
      <Link href={"/main-apps"} push asChild>
        <Button title="Get Started" />
      </Link>
    </View>
  );
}
```

Langkah 4: Mengatur navigasi main-apps

Setelah mengatur navigasi pada kontainer utama menggunakan Stack, selanjutnya membuat komponen Tab Navigation pada *main-apps/_layout.jsx* dimana seluruh file yang ada pada *main-apps* akan dibungkus menggunakan Tab Navigation.

main-apps/_layout.jsx:


```

1  import { Ionicons } from "@expo/vector-icons";
2  import Entypo from "@expo/vector-icons/Entypo";
3  import { Tabs } from "expo-router";
4  import { StatusBar } from "react-native";
5  import "react-native-reanimated";
6
7  export default function TabLayout() {
8    return (
9      <>
10     <StatusBar style="auto" barStyle={"dark-content"} hidden={false} />
11     <Tabs
12       screenOptions={{
13         headerShown: false,
14         tabBarActiveTintColor: "#49745e",
15         tabBarInactiveTintColor: "gray",
16         tabBarShowLabel: true,
17         tabBarStyle: {
18           backgroundColor: "white",
19           borderTopWidth: 2,
20           borderTopColor: "#3a5d4a",
21           height: 70,
22           paddingBottom: 5,
23         },
24         tabBarLabelStyle: {
25           fontSize: 12,
26           fontWeight: "600",
27         },
28       }}
29     >
30       <Tabs.Screen
31         name="index"
32         options={{
33           title: "Home",
34           tabBarIcon: ({ color, size }) => (
35             <Ionicons name="home" size={size} color={color} />
36           ),
37         }}
38       />
39       <Tabs.Screen
40         name="explore"
41         options={{
42           title: "Explore",
43           tabBarIcon: ({ color, size }) => (
44             <Entypo name="direction" size={size} color={color} />
45           ),
46         }}
47       />
48     </Tabs>
49   </>
50 );
51 }
52

```

main-apps/index.jsx:

Seperti yang telah disebutkan sebelumnya, bahwa file *index.jsx* akan merepresentasikan halaman Home Screen, dimana screen ini akan menggunakan tugas praktikum sebelumnya.

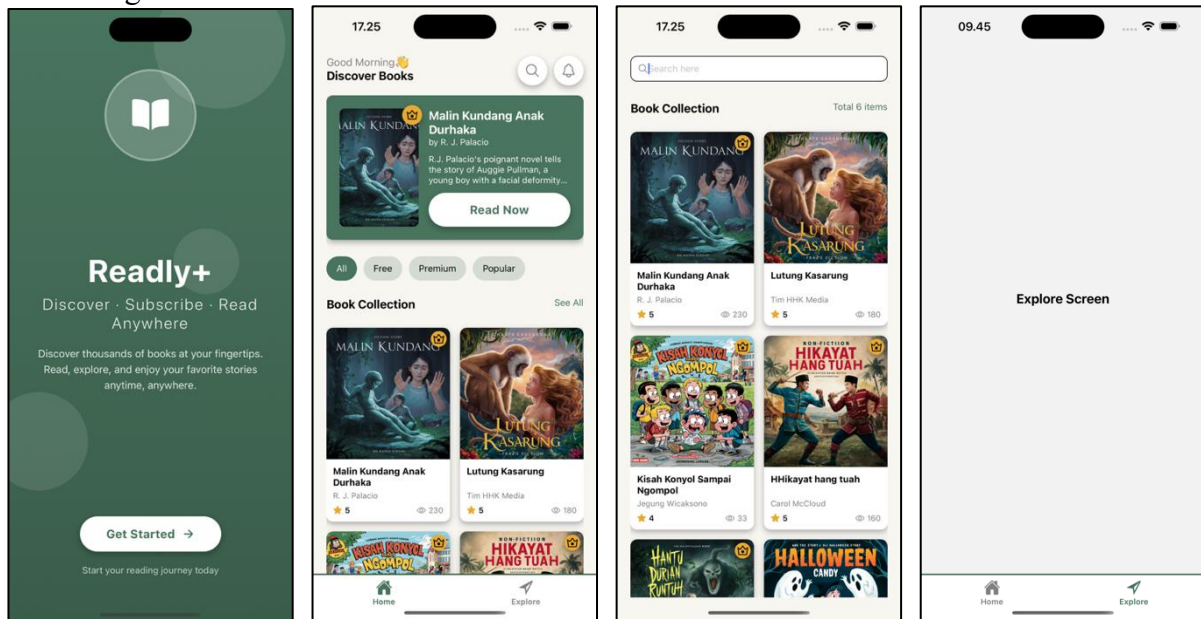
```
export { default } from "@components/module-latihan/latihan-4/praktikum";
```

main-apps/explore.jsx:

Sedangkan untuk file akan menampilkan halaman sederhana berupa sebuah Text yang bertuliskan "Halaman Explore".

Langkah 5: Output dari penggunaan Nesting Navigation

Dari penggunaan Nesting Navigation yang telah di routing pada file `_layout.jsx` maka didapatkan hasil seperti dibawah ini, dimana screen Landing Page dan Search Bar menggunakan tipe Stack Navigation sedangkan Home Screen dan Explore Screen menggunakan Tab Navigation.



Gambar 6.5 Nesting Navigation

Langkah 6: Pengiriman parameter antar Screen

Dalam contoh kasus ini, akan membuat detail halaman buku sesuai dengan parameter buku yang dikirimkan. Pada variable konstanta (*ListBook*), dimateri sebelumnya memiliki sebuah key bernama *id* sebagai penanda data unik antar item buku.

Key tersebut akan dimanfaatkan sebagai parameter antar screen pada RFC Detail.

Buatlah file RFC Detail dan masukan code seperti dibawah ini:

```
1 import { useLocalSearchParams } from "expo-router";
2 import { View, Text } from "react-native";
3 import React from "react";
4
5 export default function Detail() {
6   const { id } = useLocalSearchParams();
7   return (
8     <View>
9       <Text>ID : {id}</Text>
10    </View>
11  );
12 }
```

Pada struktur folder app panggilah RFC Detail dalam route [id].jsx

```
|— app
|   |— books
|       |— [id].jsx
```

file [id].jsx

```
export { default } from "@components/books/detail";
```

Selanjutnya, melakukan pengiriman parameter [id] pada tombol **Read More** pada CTABook:



Update file *CTABook.jsx* untuk komponen `<ButtonRead />` dengan menambahkan aksi pengiriman parameter [id] :

```
1 const ButtonRead = ({book}) => {
2   const router = useRouter();
3   const handleReadNow = () => {
4     router.push(`/books/${book.id}`);
5   };
6   return (
7     <TouchableOpacity style={styles.btn_read} onPress={handleReadNow} >
8       <Text style={styles.btn_read_text}>Read More</Text>
9     </TouchableOpacity>
10  );
11 };
```

Penjelasan:

Router Navigation

- Menggunakan *hook useRouter()* dari Expo Router
- *router* digunakan untuk navigasi antar halaman dalam aplikasi

Handler Function

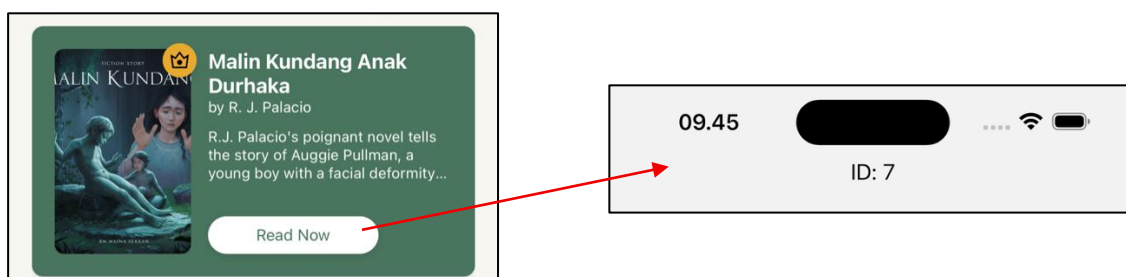
```
const handleReadNow = () => {
  router.push(`/books/${book.id}`);
};
```

- Fungsi yang dipanggil ketika tombol ditekan
- `router.push()` mengarahkan user ke halaman detail buku
- URL dinamis: `/books/[id]` - contoh: `/books/1`, `/books/2`, dll

Component Button

- `onPress={handleReadNow}`: Event handler saat tombol ditekan

Maka tampilannya akan seperti ini jika tombol Read More ditekan dan mengirimkan id ke halaman Detail:

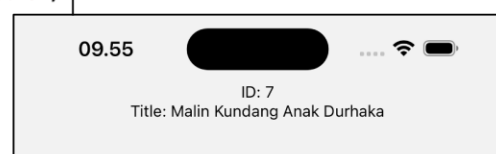


Langkah 7: Mencari data pada array object ListBook berdasarkan key id

Setelah berhasil menerima parameter `[id]` pada komponen Detail, selanjutnya ialah akan mencari item buku berdasarkan id yg diterima. Untuk mencari data dalam bentuk array object, seperti yang dimiliki oleh variable `ListBook`. Function `find()` pada javascript dapat digunakan jika ingin mencari data satuan atau satu objek. Sedangkan `filter()` mencari data dengan output array atau lebih dari satu.

Pada RFC Detail, buatlah variabel untuk menampung data item buku yang terpilih berdasarkan parameter id:

```
1 import { useLocalSearchParams } from "expo-router";
2 import { Text } from "react-native";
3 import { SafeAreaView } from "react-native-safe-area-context";
4 import { ListBook } from "../../constants/list_books";
5
6 export default function Detail() {
7   const { id } = useLocalSearchParams();
8   const book = ListBook.find((book) => book.id === id);
9
10  return (
11    <SafeAreaView style={{ flex: 1, alignItems: "center" }}>
12      <Text>ID: {id}</Text>
13      <Text>Title: {book.title}</Text>
14    </SafeAreaView>
15  );
16 }
```



Penjelasan:

Mengambil Parameter ID dari URL

```
const { id } = useLocalSearchParams();
```

- Mengambil parameter id dari URL

- Contoh: jika URL adalah /books/5, maka id = "5"

Mencari Data Buku

```
const book = ListBook.find((book) => book.id === id);
```

- Mencari buku dalam array ListBook yang memiliki id yang sesuai
- Menggunakan method find() untuk mencari item pertama yang cocok

6.4 Tugas Latihan Praktikum

Pada tugas praktikum kali ini, akan melanjutkan pengembangan aplikasi e-catalog buku yang telah dibuat sebelumnya dengan menerapkan konsep nesting navigation serta mengimplementasikan *Drawer Navigation* sebagai bagian dari struktur navigasi utama. Jika pada tahap sebelumnya navigasi difokuskan pada pengelompokan halaman melalui struktur folder bertingkat, maka pada tahap ini akan ditambahkan mekanisme navigasi berbasis *drawer* (*side menu*) untuk meningkatkan kemudahan akses antarfitur dalam aplikasi. *Drawer Navigation* akan berperan sebagai navigasi global yang menghubungkan beberapa modul, seperti halaman beranda, katalog buku, buku premium (subscription), fitur pemutar e-book, serta pencarian berbasis peta. Dengan pendekatan ini, diharapkan mampu memahami bagaimana mengombinasikan berbagai jenis navigator dalam satu aplikasi secara terstruktur, *modular*, dan *scalable*. Selain itu, praktikum ini bertujuan untuk memperkuat pemahaman mengenai hierarki layout, relasi antar navigator, serta pengelolaan alur perpindahan antarhalaman dalam arsitektur navigasi yang lebih kompleks dan mendekati implementasi aplikasi mobile nyata.

Latihan 1: Membuat struktur Drawer Navigasi

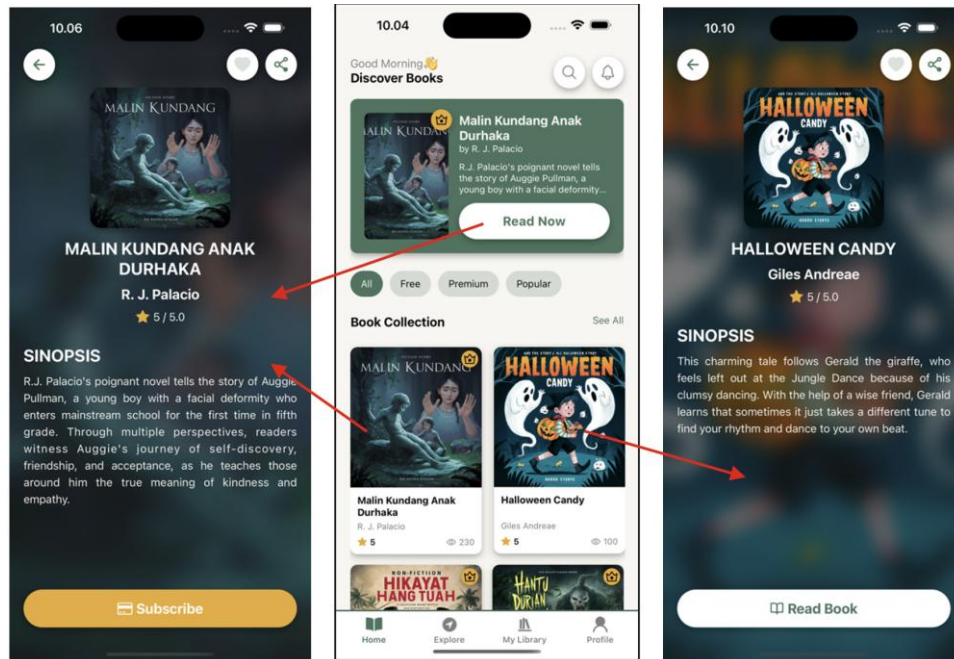
Buatlah kontainer `_layout.jsx` untuk mengatur halaman agar mekanisme menu dalam bentuk *side-menu*.

Latihan 2: Tentukan halaman apa saja yang akan dibungkus oleh Drawer

Buatlah minimal 3 buah menu pada kontainer Drawer. Masing-masing menu menampilkan informasi terkait mengenai e-catalog buku.

Latihan 3: Mengirimkan parameter antar screen

Buatlah halaman detail buku pada semua item buku yang tersedia, dan tampilkan detail informasi buku tersebut seperti contoh digambar ini:



Gambar 6.6 Tampilan Output Tugas Latihan 3

Latihan 4: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project react expo yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository **"PPB-Praktikum-NPM"**, namun commit dengan nama **"Tugas Praktikum-6"**.

Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).